Lab Experiment 7: Form Validation, Card Gallery Navigation, and AI Assistance

Table of Contents

Lab Experiment 7: Form Validation, Card Gallery Navigation, and AI Assistance 1

Learning Objectives 1

Objective 2

Lab exercises 2

Submission Requirements 7

Grading Criteria 7

Learning Objectives

By completing this lab experiment, students will be able to:

- Handle form submission using JavaScript
- Prevent default form behavior using event.preventDefault()
- Validate user input fields using JavaScript
- Display dynamic feedback and error messages in the webpage
- Use regular expressions to validate email format and password strength
- Manipulate the DOM using JavaScript event handling
- Create a horizontal card gallery with navigation controls
- Use JavaScript to dynamically update displayed content
- Implement looping navigation logic for card galleries
- Use AI tools to assist with debugging, styling, and understanding JavaScript concepts

Objective

This lab experiment introduces students to JavaScript form validation, DOM manipulation, event handling, and interactive gallery navigation.

Students will create:

1. A validated user input form
2. A card gallery with left and right navigation buttons

Students are encouraged to use AI tools to:

- Debug JavaScript errors
- Improve CSS layouts and styling
- Explain regular expressions
- Understand event handling and DOM manipulation
- Generate ideas for improving user interaction

Instructions

- Complete all class examples
- Complete both lab exercises
- Save all work in separate HTML, CSS, and JavaScript files
- Include comments in the JavaScript file explaining how AI assisted you during the activity

Lab exercises

Exercise 1: Form Validation and Event Handling with AI Assistance

Exercise Description

You will build a user input form that accepts the following details:

- Full Name
- Email Address

- Password

You will use JavaScript to:

- Prevent form submission if fields are empty or invalid
- Validate email format using a regular expression
- Validate password strength
- Display error messages below invalid fields
- Display a success message when all inputs are valid

Students may use AI assistance to help:

- Debug validation logic
- Explain regular expressions
- Improve form styling
- Troubleshoot event handling code

Learning Outcomes for Exercise 1

By completing this exercise, students will be able to:

- Handle form submission events using JavaScript
- Prevent default browser form behavior
- Validate text input fields dynamically
- Use regular expressions for input validation
- Display real-time feedback messages
- Create interactive forms using JavaScript DOM manipulation

Validation Rules Summary

- Field: name, validation rule: must not be empty
- Field: email, validation rule: must follow valid email format (name@example.com)
- Field: password, validation rule: minimum 5 characters and must contain #, $, or %

Instructions

1. Create the HTML Structure

Design a form containing:

- Name: `<input type="text">`
- Email: `<input type="text">`
- Password: `<input type="password">`
- Submit button

Add `<div>` or `<span>` elements below each field to display error messages.

2. Implement JavaScript Validation

In a separate JavaScript file:

- Listen for the submit event
- Use `event.preventDefault()`
- Validate each field
- Display error messages when invalid
- Display a success message when valid

Important Notes

- Use only JavaScript concepts covered in class
- Do not use frameworks or external libraries
- All validation must be handled using JavaScript

AI Assistance Activity for Exercise 1

Open an AI assistant and ask one or more of the following questions:

1. How does event.preventDefault() work in JavaScript?
2. How do regular expressions validate email addresses?

3. How can I display error messages dynamically in the DOM?
4. How do I validate password strength in JavaScript?

After completing the exercise, add a short comment using /* */ in the .js file describing how AI helped you.

Exercise 2: Card Gallery Navigation with AI Assistance

Exercise Description

Create a gallery of 6 cards that users can scroll through manually using Left and Right buttons.

Only 3 cards should be visible at a time.

Students may use AI assistance to help:

- Improve gallery styling
- Debug navigation logic
- Understand CSS transforms and scrolling
- Explain event listeners and DOM updates

Learning Outcomes for Exercise 2

By completing this exercise, students will be able to:

- Design a responsive card gallery using HTML and CSS
- Handle click events using JavaScript
- Dynamically update displayed content
- Implement sliding or scrolling gallery behavior
- Create looping navigation functionality
- Use DOM manipulation to control webpage layouts

Instructions

1. HTML Layout

Create a container holding 6 cards.

Each card must include:

- An image
- A title
- A short description
- A button or link

2. CSS Styling

- Arrange cards horizontally
- Display only 3 cards at a time
- Hide extra cards using overflow or transforms

3. Add Navigation Buttons

Create two buttons:

- Left (<)
- Right (>)

4. JavaScript Functionality

Use JavaScript to:

- Move to the next 3 cards when the right button is clicked
- Move to the previous 3 cards when the left button is clicked
- Create looping navigation behavior

Key Requirements

- Feature: number of cards, behavior: 6 cards total.
- Feature: visible cards, behavior: 3 cards at a time.

Important Notes

- Use only JavaScript concepts introduced in class
- Do not use external JavaScript libraries
- Use basic DOM manipulation and event handling
- Use scrollBy(), scrollTo(), or CSS transform for gallery transitions

AI Assistance Activity for Exercise 2

Open an AI assistant and ask one or more of the following questions:

1. How can I create a horizontal card gallery using CSS?
2. How does addEventListener() work for navigation buttons?
3. How do CSS transforms create sliding effects?
4. How can I loop through cards in JavaScript?

After completing the exercise, add a short reflection comment using /* */ in the .js file describing how AI assisted you during the activity.

Submission Requirements

- Upload all HTML, CSS, and JavaScript files to your GitHub repository
- Submit the GitHub repository HTTPS link

Grading Criteria

- Class examples: 5 pts
- Exercise 1: 5 pts
- Exercise 2: 5 pts